ENGENHARIA DE SOFTWARE II · TESTE DE SOFTWARE · PRÁTICA DE BDD

NewsFlow — do Gherkin ao **Cucumber**

Histórias de usuário, cenários em Gherkin e automação em Cucumber.js para a plataforma de newsletters da Vértice

Integrante	Matrícula	Papel na dupla
Eduardo Vianna Meira	844387	Negócio
Vitor Alexandre	803371	Técnico

4 histórias

12 features

52 cenários executados

276 passos — todos passando

Ferramenta: Cucumber.js · Projeto: `newsflow-bdd/`

Sumário

1. Visão geral da entrega e como executar
2. US01 · Criar newsletter (RN01)
3. US02 · Aprovar edição para envio (RN02)
4. US03 · Agendar envio (RN03)
5. US04 · Sair e voltar à lista (RN06)
6. Steps compartilhados
7. Suporte: World e domínio em memória
8. Configuração do projeto e resultado da execução

Em cada história: (1) história de usuário com critérios de aceitação, (2) features e cenários, (3) Gherkin de cada feature, (4) action words — passos reutilizáveis com seus parâmetros — e (5) código JavaScript dos steps.

1. Visão geral da entrega

Integrante	Matrícula	Papel principal
Eduardo Vianna Meira	844387	Negócio (histórias, critérios, fora de escopo)
Vitor Alexandre	803371	Técnico (steps, World, execução e relatório)

Histórias

ID	Título	Ator	Regras	Prioridade	Features
US01	Criar newsletter	Redatora	RN01	Alta	3
US02	Aprovar edição para envio	Revisor	RN02	Alta	3
US03	Agendar envio	Gestora	RN03	Alta	3
US04	Sair e voltar à lista	Assinante	RN06	Média	3

Cada feature tem 3 cenários: caminho feliz (`@feliz`), exceção (`@excecao`) e um terceiro — em geral um **Esquema do Cenário** (`@limite`) que testa o valor exatamente na fronteira e logo ao lado dela (30 segundos, 20 versões, 15 minutos, 2 reagendamentos, 5 minutos, 48 horas).

Como executar

```
npm install
npm test           # executa todos os cenários e gera relatorios/cucumber-report.html
npm run test:dry   # valida a sintaxe e confere se todo passo tem implementação
npx cucumber-js --tags "@US03 and @limite"   # só os esquemas de fronteira da US03
```

O que tem aqui

Pasta / arquivo	Conteúdo
`docs/historias.md`	As 4 histórias (US01–US04) com critérios, regras, prioridade e rastreabilidade
`docs/US0X-*.md`	Um documento por história: história, features, Gherkin, action words e JavaScript
`features/*.feature`	12 features (3 por história), todas com `# language: pt`
`features/step_definitions/`	Steps de cada história + `comum.steps.js` (passos compartilhados)
`features/support/`	`world.js` (estado por cenário) e `dominio.js` (domínio em memória, relógio simulado)
`relatorios/cucumber-report.html`	Relatório HTML da última execução

Fora de escopo

Regra	Por que ficou de fora
RN04 — Segmentação	Exige um cadastro de segmentos e critérios (plano, cidade, interesse) que seria uma história inteira; a US03 agenda o disparo sem depender disso.
RN05 — Frequência	Depende de calendário semanal e fila para a semana seguinte, o que pede várias edições já disparadas; preferimos aprofundar as fronteiras de RN03.
RN07 — Falha de entrega	É comportamento do provedor de e-mail (tentativas, *bounces*), sem um ator do negócio que perceba o valor diretamente — não caberia nos três dias de prazo.
RN08 — Notificação push	Trata de outro canal (aplicativo) com janela de silêncio e limite diário próprios; mantivemos o recorte só no e-mail.

Correções feitas na US01 (em relação à versão anterior da dupla)

- O `Esquema do Cenário` "Restaurar versão antiga depois de muitas remoções" não tinha remoções: foi renomeado para "Só as últimas 20 versões podem ser restauradas" e passou a verificar que restaurar **não apaga** as versões seguintes (critério que não era testado).
- O salvamento automático ganhou fronteira: 29 segundos → nenhuma versão; 30 → 1 versão; 60 → 1 versão.
- "1 versões salvas" virou "1 versão salva" — o step aceita singular e plural (`versão/versões salva(s)`).
- O código saiu de um único `automacao.js` e foi separado em `support/dominio.js` , `support/world.js` e um arquivo de steps por história, como pede o enunciado.
- Os passos de pré-condição (`que insiro o texto` , `que substituo`) agora falham se a ação não der certo, em vez de seguir em silêncio.

Resultado da execução

```
52 scenarios (52 passed)
276 steps (276 passed)
0m 0.080s (0m 0.012s executing your code)
```

História	Cenários	Passos	Situação
US01	13	67	todos passando
US02	11	63	todos passando
US03	17	93	todos passando
US04	11	53	todos passando

Os esquemas contam um cenário por linha de Exemplos. O `--dry-run` termina sem passos pendentes (276 passos com implementação).

2. US01 · Criar newsletter

NewsFlow (Vértice) · BDD com Cucumber.js · projeto organizado como no CucumberStudio (pasta da história → features → cenários, action words reutilizáveis e datasets nos esquemas).
Resultado da execução (`npx cucumber-js --tags @US01`): 13 cenários e 67 passos, todos passando.

1. História de usuário

Como redatora **Quero** inserir, editar e remover texto da edição **Para** montar a newsletter sem perder o que já escrevi

Critérios de aceitação

- É possível inserir texto no corpo da edição; texto em branco é recusado.
- É possível substituir um trecho existente; trecho inexistente gera aviso.
- É possível remover um trecho; corpo vazio gera aviso.
- O rascunho é salvo após 30 segundos sem digitar, gerando uma versão numerada.
- As últimas 20 versões podem ser restauradas, e restaurar uma versão não apaga as seguintes.

Regras cobertas: RN01 **Prioridade:** Alta — perda de conteúdo é a reclamação número um dos redatores.
Cenários: `features/us01-inserir-texto.feature`, `features/us01-editar-texto.feature`, `features/us01-remover-texto.feature`

2. Features e cenários

São 3 features, com 3 cenários cada (os esquemas são expandidos pelo dataset de Exemplos).

- **Feature 1: Inserir texto na edição** (`features/us01-inserir-texto.feature`)
  - `@feliz` Cenário: Inserir texto no corpo vazio
  - `@excecao` Cenário: Inserir texto em branco
  - `@limite` Esquema do Cenário: Texto inserido é salvo após 30 segundos sem digitar (dataset com 3 exemplos)
- **Feature 2: Editar texto da edição** (`features/us01-editar-texto.feature`)
  - `@feliz` Cenário: Substituir um trecho do texto
  - `@excecao` Cenário: Substituir um trecho que não existe
  - `@salvamento` Cenário: Edição gera uma nova versão numerada
- **Feature 3: Remover texto e restaurar versões** (`features/us01-remover-texto.feature`)
  - `@feliz` Cenário: Remover um trecho do texto
  - `@excecao` Cenário: Remover texto de um corpo vazio
  - `@limite` Esquema do Cenário: Só as últimas 20 versões podem ser restauradas (dataset com 3 exemplos)

3. Gherkin

Feature 1 · Inserir texto na edição

```
# language: pt
@US01
Funcionalidade: Inserir texto na edição
  Como redatora
  Quero inserir texto no corpo da edição
  Para começar a escrever a newsletter sem perder o que já escrevi

  Contexto:
    Dado que estou autenticada como redatora
    E que existe a edição "Informativo #43" em rascunho com o corpo vazio

  @feliz
  Cenário: Inserir texto no corpo vazio
    Quando insiro o texto "Olá, assinantes da Vértice!"
    Então o corpo da edição deve ser "Olá, assinantes da Vértice!"

  @excecao
  Cenário: Inserir texto em branco
    Quando insiro o texto "   "
    Então o corpo da edição deve continuar vazio
    E devo ver a mensagem "Digite algum texto para inserir"

  @limite
  Esquema do Cenário: Texto inserido é salvo após 30 segundos sem digitar
    Dado que insiro o texto "Primeiro parágrafo"
    Quando eu espero <segundos> segundos sem digitar
    Então o rascunho deve ter <versoes> versões salvas

    Exemplos:
      | segundos | versoes |
      | 29       | 0       |
      | 30       | 1       |
      | 60       | 1       |
```

Feature 2 · Editar texto da edição

```
# language: pt
@US01
Funcionalidade: Editar texto da edição
  Como redatora
  Quero alterar o texto que já escrevi
  Para corrigir e melhorar a newsletter

  Contexto:
    Dado que estou autenticada como redatora
    E que existe a edição "Informativo #43" em rascunho com o corpo "Olá, assinantes!"

  @feliz
  Cenário: Substituir um trecho do texto
    Quando substituo "assinantes" por "leitores"
    Então o corpo da edição deve ser "Olá, leitores!"

  @excecao
  Cenário: Substituir um trecho que não existe
    Quando substituo "clientes" por "leitores"
    Então o corpo da edição deve continuar "Olá, assinantes!"
```

```
    E devo ver a mensagem "Trecho não encontrado"

  @salvamento
  Cenário: Edição gera uma nova versão numerada
    Dado que a edição já tem 1 versão salva
    E que substituo "assinantes" por "leitores"
    Quando eu espero 30 segundos sem digitar
    Então o rascunho deve ter 2 versões salvas
    E a versão 2 deve ter o corpo "Olá, leitores!"
```

Feature 3 · Remover texto e restaurar versões

```
# language: pt
@US01
Funcionalidade: Remover texto e restaurar versões
  Como redatora
  Quero remover trechos do texto
  Para tirar o que não serve mais, podendo voltar atrás por uma versão salva

  Contexto:
    Dado que estou autenticada como redatora
    E que existe a edição "Informativo #43" em rascunho com o corpo "Olá, assinantes! Aproveite a
oferta."

  @feliz
  Cenário: Remover um trecho do texto
    Quando removo o trecho " Aproveite a oferta."
    Então o corpo da edição deve ser "Olá, assinantes!"

  @excecao
  Cenário: Remover texto de um corpo vazio
    Dado que o corpo da edição está vazio
    Quando removo o trecho "oferta"
    Então devo ver a mensagem "Não há texto para remover"

  @limite
  Esquema do Cenário: Só as últimas 20 versões podem ser restauradas
    Dado que a edição já tem <salvas> versões salvas
    Quando restauro a versão <versao>
    Então devo ver a mensagem "<mensagem>"
    E o rascunho deve ter 20 versões salvas

    Exemplos:
      | salvas | versao | mensagem                          |
      | 20     | 1      | Versão restaurada                 |
      | 21     | 2      | Versão restaurada                 |
      | 21     | 1      | Versão indisponível para restauro |
```

4. Action words (passos reutilizáveis)

Dado:

- `que estou autenticada como redatora`
- `que existe a edição "{titulo}" em rascunho com o corpo vazio` — String
- `que existe a edição "{titulo}" em rascunho com o corpo "{corpo}"` — String, String
- `que o corpo da edição está vazio`
- `que a edição já tem {quantidade} versão/versões salva(s)` — Integer

- `que insiro o texto "{texto}"` — String
- `que substituo "{trecho}" por "{novo}"` — String, String

Quando:

- `insiro o texto "{texto}"` — String
- `substituo "{trecho}" por "{novo}"` — String, String
- `removo o trecho "{trecho}"` — String
- `eu espero {segundos} segundos sem digitar` — Integer
- `restauro a versão {numero}` — Integer

Então:

- `o corpo da edição deve ser "{esperado}"` — String
- `o corpo da edição deve continuar "{esperado}"` — String
- `o corpo da edição deve continuar vazio`
- `o rascunho deve ter {quantidade} versão/versões salva(s)` — Integer
- `a versão {numero} deve ter o corpo "{corpo}"` — Integer, String

Reutilizadas de outras histórias / comuns:

- `devo ver a mensagem "{mensagem}"` — comum.steps.js

Observação: o Cucumber diferencia Dado de Quando; por isso uma ação usada como pré-condição (ex.: "que insiro o texto") também existe como `Given`.

5. Código JavaScript (Cucumber.js)

features/step_definitions/us01-criar-newsletter.steps.js

Steps da história. O estado fica no World (`this`), recriado a cada cenário; o domínio em memória e o relógio simulado (sem `sleep`) estão em `features/support/dominio.js` e `features/support/world.js` (Parte 7 deste documento).

```
// ============================================================================
// US01 · Criar newsletter (RN01) — inserir, editar, remover e restaurar
// ============================================================================
const { Given, When, Then } = require('@cucumber/cucumber')
const assert = require('node:assert/strict')

// ---------------------------- DADO ----------------------------
Given('que estou autenticada como redatora', function () {
  this.servico.autenticar('redatora', 'Carla')
})

Given('que existe a edição {string} em rascunho com o corpo vazio', function (titulo) {
  this.edicao = this.servico.criarEdicao(titulo, '', 'Carla')
})

Given('que existe a edição {string} em rascunho com o corpo {string}', function (titulo, corpo) {
  this.edicao = this.servico.criarEdicao(titulo, corpo, 'Carla')
})
```

```
Given('que o corpo da edição está vazio', function () {
  this.edicao.corpo = ''
})

Given('que a edição já tem {int} versão/versões salva(s)', function (quantidade) {
  this.servico.definirVersoes(this.edicao, quantidade)
})

// Mesmas ações usadas como pré-condição (o Cucumber separa Dado de Quando)
Given('que insiro o texto {string}', function (texto) {
  assert.ok(this.servico.inserirTexto(this.edicao, texto).ok)
})

Given('que substituo {string} por {string}', function (trecho, novo) {
  assert.ok(this.servico.substituirTrecho(this.edicao, trecho, novo).ok)
})

// ---------------------------- QUANDO ----------------------------
When('insiro o texto {string}', function (texto) {
  this.resultado = this.servico.inserirTexto(this.edicao, texto)
})

When('substituo {string} por {string}', function (trecho, novo) {
  this.resultado = this.servico.substituirTrecho(this.edicao, trecho, novo)
})

When('removo o trecho {string}', function (trecho) {
  this.resultado = this.servico.removerTrecho(this.edicao, trecho)
})

When('eu espero {int} segundos sem digitar', function (segundos) {
  this.avancarSegundos(segundos)
})

When('restauro a versão {int}', function (numero) {
  this.resultado = this.servico.restaurarVersao(this.edicao, numero)
})

// ---------------------------- ENTÃO ----------------------------
Then('o corpo da edição deve ser {string}', function (esperado) {
  assert.equal(this.edicao.corpo, esperado)
})

Then('o corpo da edição deve continuar {string}', function (esperado) {
  assert.equal(this.edicao.corpo, esperado)
})

Then('o corpo da edição deve continuar vazio', function () {
  assert.equal(this.edicao.corpo, '')
})

Then('o rascunho deve ter {int} versão/versões salva(s)', function (quantidade) {
  assert.equal(this.edicao.versoes.length, quantidade)
})

Then('a versão {int} deve ter o corpo {string}', function (numero, corpo) {
  const versao = this.edicao.versoes.find((v) => v.numero === numero)
  assert.ok(versao, `A versão ${numero} não existe`)
  assert.equal(versao.corpo, corpo)
})
```

3. US02 · Aprovar edição para envio

NewsFlow (Vértice) · BDD com Cucumber.js · projeto organizado como no CucumberStudio (pasta da história → features → cenários, action words reutilizáveis e datasets nos esquemas).
Resultado da execução (`npx cucumber-js --tags @US02`): 11 cenários e 63 passos, todos passando.

1. História de usuário

Como revisor **Quero** aprovar ou devolver com comentário a edição escrita por outra pessoa **Para** garantir que nada sai sem conferência, sem virar gargalo da equipe

Critérios de aceitação

- Um revisor aprova uma edição que está aguardando revisão.
- O autor não aprova a própria edição, mesmo tendo papel de revisor; quem não é revisor também não aprova.
- A reprovação devolve a edição ao rascunho e exige um comentário, que fica visível para a redatora.
- Só edições aprovadas podem ser agendadas.

Regras cobertas: RN02 **Prioridade:** Alta — sem aprovação, um erro de texto chega a 40 mil assinantes.
Cenários: `features/us02-aprovar-edicao.feature`, `features/us02-reprovar-edicao.feature`, `features/us02-envio-exige-aprovacao.feature`

2. Features e cenários

São 3 features, com 3 cenários cada (os esquemas são expandidos pelo dataset de Exemplos).

- **Feature 1: Aprovar edição** (`features/us02-aprovar-edicao.feature`)
  - `@feliz` Cenário: Revisor aprova a edição de outra pessoa
  - `@excecao` Cenário: Autora não pode aprovar a própria edição
  - `@excecao` Cenário: Quem não é revisor não pode aprovar
- **Feature 2: Reprovar edição com comentário** (`features/us02-reprovar-edicao.feature`)
  - `@feliz` Cenário: Reprovar com comentário devolve a edição ao rascunho
  - `@excecao` Cenário: Reprovar sem comentário é recusado
  - `@excecao` Cenário: Edição já aprovada não pode ser reprovada
- **Feature 3: Envio exige aprovação de um revisor** (`features/us02-envio-exige-aprovacao.feature`)
  - `@feliz` Cenário: Edição aprovada pode ser agendada
  - `@excecao` Cenário: Edição reprovada não pode ser agendada
  - `@limite` Esquema do Cenário: Só a situação "aprovada" libera o agendamento (dataset com 3 exemplos)

3. Gherkin

Feature 1 · Aprovar edição

```
# language: pt
@US02
Funcionalidade: Aprovar edição
  Como revisor
  Quero aprovar uma edição escrita por outra pessoa
  Para liberar o envio sem virar gargalo

  Contexto:
    Dado que a edição "Informativo #42" foi escrita pela redatora "Carla"
    E que a edição está aguardando revisão

  @feliz
  Cenário: Revisor aprova a edição de outra pessoa
    Dado que estou autenticado como revisor "Rafael"
    Quando aprovo a edição
    Então a edição deve estar "aprovada"
    E devo ver a mensagem "Edição aprovada"

  @excecao
  Cenário: Autora não pode aprovar a própria edição
    Dado que estou autenticada como revisora "Carla"
    Quando aprovo a edição
    Então a edição deve continuar "aguardando revisão"
    E devo ver a mensagem "O autor não pode aprovar a própria edição"

  @excecao
  Cenário: Quem não é revisor não pode aprovar
    Dado que estou autenticada como redatora "Beatriz"
    Quando aprovo a edição
    Então a edição deve continuar "aguardando revisão"
    E devo ver a mensagem "Somente revisores podem aprovar edições"
```

Feature 2 · Reprovar edição com comentário

```
# language: pt
@US02
Funcionalidade: Reprovar edição com comentário
  Como revisor
  Quero devolver a edição ao rascunho explicando o que corrigir
  Para que a redatora saiba exatamente o que mudar sem precisar me procurar

  Contexto:
    Dado que a edição "Informativo #42" foi escrita pela redatora "Carla"
    E que estou autenticado como revisor "Rafael"

  @feliz
  Cenário: Reprovar com comentário devolve a edição ao rascunho
    Dado que a edição está aguardando revisão
    Quando reprovo a edição com o comentário "Corrigir o link da oferta"
    Então a edição deve estar "em rascunho"
    E a redatora "Carla" deve ver o comentário "Corrigir o link da oferta"

  @excecao
  Cenário: Reprovar sem comentário é recusado
    Dado que a edição está aguardando revisão
    Quando reprovo a edição sem comentário
```

```
    Então a edição deve continuar "aguardando revisão"
    E devo ver a mensagem "O comentário é obrigatório para reprovar"

  @excecao
  Cenário: Edição já aprovada não pode ser reprovada
    Dado que a edição foi aprovada pelo revisor "Paula"
    Quando reprovo a edição com o comentário "Trocar a imagem de capa"
    Então a edição deve continuar "aprovada"
    E devo ver a mensagem "A edição não está aguardando revisão"
```

Feature 3 · Envio exige aprovação de um revisor

```
# language: pt
@US02
Funcionalidade: Envio exige aprovação de um revisor
  Como gestora de comunicação
  Quero que só edições aprovadas possam ser agendadas
  Para nunca disparar para 40 mil assinantes um texto que ninguém conferiu

  Contexto:
    Dado que a edição "Informativo #42" foi escrita pela redatora "Carla"
    E que estou autenticada como gestora

  @feliz
  Cenário: Edição aprovada pode ser agendada
    Dado que a edição foi aprovada pelo revisor "Rafael"
    Quando agendo o envio para daqui a 30 minutos
    Então o disparo deve estar agendado para daqui a 30 minutos
    E devo ver a mensagem "Envio agendado"

  @excecao
  Cenário: Edição reprovada não pode ser agendada
    Dado que a edição foi reprovada pelo revisor "Rafael" com o comentário "Revisar o título"
    Quando agendo o envio para daqui a 30 minutos
    Então o disparo não deve estar agendado
    E devo ver a mensagem "A edição precisa ser aprovada por um revisor"

  @limite
  Esquema do Cenário: Só a situação "aprovada" libera o agendamento
    Dado que a edição está "<situacao>"
    Quando agendo o envio para daqui a 30 minutos
    Então devo ver a mensagem "<mensagem>"

    Exemplos:
      | situacao           | mensagem                                    |
      | em rascunho        | A edição precisa ser aprovada por um revisor |
      | aguardando revisão | A edição precisa ser aprovada por um revisor |
      | aprovada           | Envio agendado                              |
```

4. Action words (passos reutilizáveis)

Dado:

- `que a edição "{titulo}" foi escrita pela redatora "{autora}"` — String, String
- `que a edição está aguardando revisão`
- `que a edição está "{situacao}"` — String
- `que estou autenticado/autenticada como revisor/revisora "{nome}"` — String
- `que estou autenticada como redatora "{nome}"` — String

- `que a edição foi aprovada pelo revisor "{revisor}"` — String
- `que a edição foi reprovada pelo revisor "{revisor}" com o comentário "{comentario}"` — String, String

Quando:

- `aprovo a edição`
- `reprovo a edição com o comentário "{comentario}"` — String
- `reprovo a edição sem comentário`

Então:

- `a edição deve estar "{situacao}"` — String
- `a edição deve continuar "{situacao}"` — String
- `a redatora "{autora}" deve ver o comentário "{comentario}"` — String, String

Reutilizadas de outras histórias / comuns:

- `que estou autenticada como gestora` — comum.steps.js
- `devo ver a mensagem "{mensagem}"` — comum.steps.js
- `agendo o envio para daqui a {minutos} minutos` — US03
- `o disparo deve estar agendado para daqui a {minutos} minutos` — US03
- `o disparo não deve estar agendado` — US03

Observação: o Cucumber diferencia Dado de Quando; por isso uma ação usada como pré-condição (ex.: "que insiro o texto") também existe como `Given`.

5. Código JavaScript (Cucumber.js)

features/step_definitions/us02-aprovar-edicao.steps.js

Steps da história. O estado fica no World (`this`), recriado a cada cenário; o domínio em memória e o relógio simulado (sem `sleep`) estão em `features/support/dominio.js` e `features/support/world.js` (Parte 7 deste documento).

```
// ============================================================================
// US02 · Aprovar edição para envio (RN02)
// ============================================================================
const { Given, When, Then } = require('@cucumber/cucumber')
const assert = require('node:assert/strict')
const { SITUACAO } = require('../support/dominio')

// Executa uma ação como outro usuário e devolve a sessão a quem estava logado
function comoRevisor (mundo, nome, acao) {
  const sessaoAnterior = mundo.servico.usuario
  mundo.servico.autenticar('revisor', nome)
  const resultado = acao()
  mundo.servico.usuario = sessaoAnterior
  assert.ok(resultado.ok, resultado.mensagem)
}

// ---------------------------- DADO ----------------------------
Given('que a edição {string} foi escrita pela redatora {string}', function (titulo, autora) {
```

```
  this.edicao = this.servico.criarEdicao(titulo, 'Novidades da semana', autora)
})

Given('que a edição está aguardando revisão', function () {
  this.servico.enviarParaRevisao(this.edicao)
})

Given('que a edição está {string}', function (situacao) {
  assert.ok(Object.values(SITUACAO).includes(situacao), `Situação desconhecida: ${situacao}`)
  this.edicao.situacao = situacao
})

Given('que estou autenticado/autenticada como revisor/revisora {string}', function (nome) {
  this.servico.autenticar('revisor', nome)
})

Given('que estou autenticada como redatora {string}', function (nome) {
  this.servico.autenticar('redatora', nome)
})

Given('que a edição foi aprovada pelo revisor {string}', function (revisor) {
  this.servico.enviarParaRevisao(this.edicao)
  comoRevisor(this, revisor, () => this.servico.aprovar(this.edicao))
})

Given('que a edição foi reprovada pelo revisor {string} com o comentário {string}', function (revisor,
comentario) {
  this.servico.enviarParaRevisao(this.edicao)
  comoRevisor(this, revisor, () => this.servico.reprovar(this.edicao, comentario))
})

// ---------------------------- QUANDO ----------------------------
When('aprovo a edição', function () {
  this.resultado = this.servico.aprovar(this.edicao)
})

When('reprovo a edição com o comentário {string}', function (comentario) {
  this.resultado = this.servico.reprovar(this.edicao, comentario)
})

When('reprovo a edição sem comentário', function () {
  this.resultado = this.servico.reprovar(this.edicao, '')
})

// ---------------------------- ENTÃO ----------------------------
Then('a edição deve estar {string}', function (situacao) {
  assert.equal(this.edicao.situacao, situacao)
})

Then('a edição deve continuar {string}', function (situacao) {
  assert.equal(this.edicao.situacao, situacao)
})

Then('a redatora {string} deve ver o comentário {string}', function (autora, comentario) {
  assert.equal(this.edicao.autor, autora)
  assert.ok(
    this.edicao.comentarios.some((c) => c.texto === comentario),
    `Comentário "${comentario}" não encontrado na edição`
  )
})
```

4. US03 · Agendar envio

NewsFlow (Vértice) · BDD com Cucumber.js · projeto organizado como no CucumberStudio (pasta da história → features → cenários, action words reutilizáveis e datasets nos esquemas).
Resultado da execução (`npx cucumber-js --tags @US03`): 17 cenários e 93 passos, todos passando.

1. História de usuário

Como gestora de comunicação **Quero** agendar, reagendar e cancelar o envio de uma edição aprovada **Para** que ela chegue no melhor horário sem eu precisar apertar o botão na hora

Critérios de aceitação

- O envio é agendado com no mínimo 15 minutos de antecedência, no fuso `America/Sao_Paulo`.
- Um envio pode ser reagendado no máximo 2 vezes, respeitando os mesmos 15 minutos.
- Um envio pode ser cancelado até 5 minutos antes do horário marcado.
- Tentativas fora dessas regras mantêm o agendamento como estava e informam o motivo.

Regras cobertas: RN03 (e RN02, que exige edição aprovada) **Prioridade:** Alta — é a fronteira numérica mais importante do sistema e onde os erros custam mais. **Cenários:** `features/us03-agendar-envio.feature`, `features/us03-reagendar-envio.feature`, `features/us03-cancelar-envio.feature`

2. Features e cenários

São 3 features, com 3 cenários cada (os esquemas são expandidos pelo dataset de Exemplos).

- **Feature 1: Agendar envio de newsletter** (`features/us03-agendar-envio.feature`)
  - `@feliz` Cenário: Agendamento com antecedência suficiente
  - `@excecao` Cenário: Agendamento com menos de 15 minutos é recusado
  - `@limite` Esquema do Cenário: Antecedência mínima de 15 minutos para agendar (dataset com 5 exemplos)
- **Feature 2: Reagendar envio** (`features/us03-reagendar-envio.feature`)
  - `@feliz` Cenário: Reagendar pela primeira vez
  - `@excecao` Cenário: Reagendar para menos de 15 minutos é recusado
  - `@limite` Esquema do Cenário: Limite de 2 reagendamentos (dataset com 3 exemplos)
- **Feature 3: Cancelar envio agendado** (`features/us03-cancelar-envio.feature`)
  - `@feliz` Cenário: Cancelar com antecedência
  - `@excecao` Cenário: Cancelar faltando menos de 5 minutos é recusado
  - `@limite` Esquema do Cenário: Cancelamento permitido até 5 minutos antes (dataset com 3 exemplos)

3. Gherkin

Feature 1 · Agendar envio de newsletter

```gherkin
# language: pt
@US03
Funcionalidade: Agendar envio de newsletter
  Como gestora de comunicação
  Quero agendar o envio de uma edição aprovada
  Para que ela chegue no melhor horário sem eu precisar apertar o botão

  Contexto:
    Dado que estou autenticada como gestora
    E que existe a edição "Informativo #42" aprovada

  @feliz
  Cenário: Agendamento com antecedência suficiente
    Quando agendo o envio para daqui a 30 minutos
    Então o disparo deve estar agendado para daqui a 30 minutos
    E o disparo deve usar o fuso "America/Sao_Paulo"
    E devo ver a mensagem "Envio agendado"

  @excecao
  Cenário: Agendamento com menos de 15 minutos é recusado
    Quando agendo o envio para daqui a 10 minutos
    Então o disparo não deve estar agendado
    E devo ver a mensagem "O envio precisa de ao menos 15 minutos"

  @limite
  Esquema do Cenário: Antecedência mínima de 15 minutos para agendar
    Quando agendo o envio para daqui a <minutos> minutos
    Então devo ver a mensagem "<mensagem>"

    Exemplos:
      | minutos | mensagem                              |
      | 60      | Envio agendado                        |
      | 16      | Envio agendado                        |
      | 15      | Envio agendado                        |
      | 14      | O envio precisa de ao menos 15 minutos |
      | 0       | O envio precisa de ao menos 15 minutos |
```

Feature 2 · Reagendar envio

```gherkin
# language: pt
@US03
Funcionalidade: Reagendar envio
  Como gestora de comunicação
  Quero mudar o horário de um envio já agendado
  Para acompanhar mudanças de última hora sem refazer a edição

  Contexto:
    Dado que estou autenticada como gestora
    E que existe a edição "Informativo #42" aprovada
    E que o envio está agendado para daqui a 60 minutos

  @feliz
  Cenário: Reagendar pela primeira vez
    Quando reagendo o envio para daqui a 120 minutos
    Então o disparo deve estar agendado para daqui a 120 minutos
    E devo ver a mensagem "Envio reagendado"
```

```
  @excecao
  Cenário: Reagendar para menos de 15 minutos é recusado
    Quando reagendo o envio para daqui a 10 minutos
    Então o disparo deve continuar agendado para daqui a 60 minutos
    E devo ver a mensagem "O envio precisa de ao menos 15 minutos"

  @limite
  Esquema do Cenário: Limite de 2 reagendamentos
    Dado que o envio já foi reagendado <vezes>
    Quando reagendo o envio para daqui a 120 minutos
    Então devo ver a mensagem "<mensagem>"

    Exemplos:
      | vezes    | mensagem                              |
      | 0 vezes  | Envio reagendado                      |
      | 1 vez    | Envio reagendado                      |
      | 2 vezes  | O envio só pode ser reagendado 2 vezes |
```

Feature 3 · Cancelar envio agendado

```
# language: pt
@US03
Funcionalidade: Cancelar envio agendado
  Como gestora de comunicação
  Quero cancelar um envio que não deve mais sair
  Para não mandar aos assinantes uma informação que ficou errada

  Contexto:
    Dado que estou autenticada como gestora
    E que existe a edição "Informativo #42" aprovada
    E que o envio está agendado para daqui a 60 minutos

  @feliz
  Cenário: Cancelar com antecedência
    Dado que faltam 30 minutos para o envio
    Quando cancelo o envio
    Então o disparo deve estar cancelado
    E devo ver a mensagem "Envio cancelado"

  @excecao
  Cenário: Cancelar faltando menos de 5 minutos é recusado
    Dado que faltam 2 minutos para o envio
    Quando cancelo o envio
    Então o disparo deve continuar agendado
    E devo ver a mensagem "O cancelamento só é permitido até 5 minutos antes do envio"

  @limite
  Esquema do Cenário: Cancelamento permitido até 5 minutos antes
    Dado que faltam <minutos> minutos para o envio
    Quando cancelo o envio
    Então devo ver a mensagem "<mensagem>"

    Exemplos:
      | minutos | mensagem                                              |
      | 6       | Envio cancelado                                       |
      | 5       | Envio cancelado                                       |
      | 4       | O cancelamento só é permitido até 5 minutos antes do envio |
```

4. Action words (passos reutilizáveis)

Dado:

- `que existe a edição "{titulo}" aprovada` — String
- `que o envio está agendado para daqui a {minutos} minutos` — Integer
- `que o envio já foi reagendado {vezes} vez(es)` — Integer
- `que faltam {minutos} minutos para o envio` — Integer

Quando:

- `agendo o envio para daqui a {minutos} minutos` — Integer
- `reagendo o envio para daqui a {minutos} minutos` — Integer
- `cancelo o envio`

Então:

- `o disparo deve estar agendado para daqui a {minutos} minutos` — Integer
- `o disparo deve continuar agendado para daqui a {minutos} minutos` — Integer
- `o disparo deve usar o fuso "{fuso}"` — String
- `o disparo não deve estar agendado`
- `o disparo deve estar cancelado`
- `o disparo deve continuar agendado`

Reutilizadas de outras histórias / comuns:

- `que estou autenticada como gestora` — comum.steps.js
- `devo ver a mensagem "{mensagem}"` — comum.steps.js

Observação: o Cucumber diferencia Dado de Quando; por isso uma ação usada como pré-condição (ex.: "que insiro o texto") também existe como `Given`.

5. Código JavaScript (Cucumber.js)

features/step_definitions/us03-agendar-envio.steps.js

Steps da história. O estado fica no World (`this`), recriado a cada cenário; o domínio em memória e o relógio simulado (sem `sleep`) estão em `features/support/dominio.js` e `features/support/world.js` (Parte 7 deste documento).

```
// ============================================================================
// US03 · Agendar envio (RN03) — agendar, reagendar e cancelar
// ============================================================================
const { Given, When, Then } = require('@cucumber/cucumber')
const assert = require('node:assert/strict')
const { SITUACAO, MINUTO } = require('../support/dominio')

// ---------------------------- DADO ----------------------------
Given('que existe a edição {string} aprovada', function (titulo) {
  this.edicao = this.servico.criarEdicao(titulo, 'Novidades da semana', 'Carla')
  this.edicao.situacao = SITUACAO.APROVADA
```

```
  this.edicao.aprovadaPor = 'Rafael'
})

Given('que o envio está agendado para daqui a {int} minutos', function (minutos) {
  const r = this.servico.agendar(this.edicao, minutos)
  assert.ok(r.ok, r.mensagem)
})

Given('que o envio já foi reagendado {int} vez(es)', function (vezes) {
  this.edicao.disparo.reagendamentos = vezes
})

Given('que faltam {int} minutos para o envio', function (minutos) {
  const faltamAgora = this.servico.minutosAteODisparo(this.edicao)
  this.avancarMinutos(faltamAgora - minutos)
})

// ---------------------------- QUANDO ----------------------------
When('agendo o envio para daqui a {int} minutos', function (minutos) {
  this.resultado = this.servico.agendar(this.edicao, minutos)
})

When('reagendo o envio para daqui a {int} minutos', function (minutos) {
  this.resultado = this.servico.reagendar(this.edicao, minutos)
})

When('cancelo o envio', function () {
  this.resultado = this.servico.cancelar(this.edicao)
})

// ---------------------------- ENTÃO ----------------------------
function assertAgendadoPara (mundo, minutos) {
  const d = mundo.edicao.disparo
  assert.ok(d, 'O disparo não foi agendado')
  assert.equal(d.situacao, 'agendado')
  assert.equal(d.horario, mundo.servico.agora + minutos * MINUTO)
}

Then('o disparo deve estar agendado para daqui a {int} minutos', function (minutos) {
  assertAgendadoPara(this, minutos)
})

Then('o disparo deve continuar agendado para daqui a {int} minutos', function (minutos) {
  assertAgendadoPara(this, minutos)
})

Then('o disparo deve usar o fuso {string}', function (fuso) {
  assert.equal(this.edicao.disparo.fuso, fuso)
})

Then('o disparo não deve estar agendado', function () {
  assert.equal(this.edicao.disparo, null)
})

Then('o disparo deve estar cancelado', function () {
  assert.equal(this.edicao.disparo.situacao, 'cancelado')
})

Then('o disparo deve continuar agendado', function () {
  assert.equal(this.edicao.disparo.situacao, 'agendado')
})
```

5. US04 · Sair e voltar à lista

NewsFlow (Vértice) · BDD com Cucumber.js · projeto organizado como no CucumberStudio (pasta da história → features → cenários, action words reutilizáveis e datasets nos esquemas).
Resultado da execução (`npx cucumber-js --tags @US04`): 11 cenários e 53 passos, todos passando.

1. História de usuário

Como assinante **Quero** sair da lista pelo link do e-mail e poder voltar quando quiser, confirmando por e-mail **Para** receber só o que eu aceito, sem ninguém me reinscrever sem meu consentimento

Critérios de aceitação

- Todo e-mail traz o link de descadastro.
- O descadastro tem efeito imediato e é confirmado por e-mail.
- O retorno à lista exige duplo opt-in: o link de confirmação vale por 48 horas e só pode ser usado uma vez.
- Quem está fora da lista — inclusive quem pediu retorno e ainda não confirmou — não recebe disparos.

Regras cobertas: RN06 **Prioridade:** Média — não trava o fluxo editorial, mas é obrigação legal (LGPD) e protege a reputação da lista. **Cenários:** `features/us04-descadastrar.feature`, `features/us04-voltar-a-lista.feature`, `features/us04-disparo-respeita-descadastro.feature`

2. Features e cenários

São 3 features, com 3 cenários cada (os esquemas são expandidos pelo dataset de Exemplos).

- **Feature 1: Descadastrar pelo link do e-mail** (`features/us04-descadastrar.feature`)
  - `@feliz` Cenário: Descadastro tem efeito imediato e é confirmado por e-mail
  - `@excecao` Cenário: Quem já saiu da lista não é descadastrado de novo
  - `@feliz` Cenário: Toda newsletter traz o link de descadastro
- **Feature 2: Voltar à lista com duplo opt-in** (`features/us04-voltar-a-lista.feature`)
  - `@feliz` Cenário: Pedido de retorno envia link de confirmação e ainda não reinscreve
  - `@excecao` Cenário: Link de confirmação não pode ser usado duas vezes
  - `@limite` Esquema do Cenário: Link de confirmação vale por 48 horas (dataset com 3 exemplos)
- **Feature 3: Disparo respeita o descadastro** (`features/us04-disparo-respeita-descadastro.feature`)
  - `@feliz` Cenário: Quem está na lista recebe a edição
  - `@excecao` Cenário: Quem se descadastrou não recebe a edição
  - `@excecao` Cenário: Retorno ainda não confirmado não volta a receber

3. Gherkin

Feature 1 · Descadastrar pelo link do e-mail

```gherkin
# language: pt
@US04
Funcionalidade: Descadastrar pelo link do e-mail
  Como assinante
  Quero sair da lista pelo link que vem em todo e-mail
  Para parar de receber newsletters na hora em que eu decidir

  Contexto:
    Dado que a assinante "ana@exemplo.com" está na lista

  @feliz
  Cenário: Descadastro tem efeito imediato e é confirmado por e-mail
    Quando a assinante "ana@exemplo.com" se descadastra pelo link do e-mail
    Então a assinante "ana@exemplo.com" deve estar fora da lista
    E a assinante "ana@exemplo.com" deve receber o e-mail "Confirmação de descadastro"

  @excecao
  Cenário: Quem já saiu da lista não é descadastrado de novo
    Dado que a assinante "ana@exemplo.com" se descadastrou
    Quando a assinante "ana@exemplo.com" se descadastra pelo link do e-mail
    Então devo ver a mensagem "Você já está fora da lista"
    E a assinante "ana@exemplo.com" deve ter recebido 1 e-mail "Confirmação de descadastro"

  @feliz
  Cenário: Toda newsletter traz o link de descadastro
    Dado que existe a edição "Informativo #42" aprovada
    Quando a edição "Informativo #42" é disparada
    Então o e-mail "Informativo #42" recebido por "ana@exemplo.com" deve trazer o link de descadastro
```

Feature 2 · Voltar à lista com duplo opt-in

```gherkin
# language: pt
@US04
Funcionalidade: Voltar à lista com duplo opt-in
  Como assinante que saiu da lista
  Quero voltar a receber as newsletters confirmando pelo meu e-mail
  Para que ninguém consiga me inscrever de volta sem o meu consentimento

  Contexto:
    Dado que a assinante "ana@exemplo.com" se descadastrou

  @feliz
  Cenário: Pedido de retorno envia link de confirmação e ainda não reinscreve
    Quando a assinante "ana@exemplo.com" pede para voltar à lista
    Então a assinante "ana@exemplo.com" deve receber o e-mail "Confirme seu retorno à lista"
    E a assinante "ana@exemplo.com" deve continuar fora da lista

  @excecao
  Cenário: Link de confirmação não pode ser usado duas vezes
    Dado que a assinante "ana@exemplo.com" pediu para voltar à lista há 1 hora
    E que a assinante "ana@exemplo.com" já confirmou o retorno
    Quando a assinante "ana@exemplo.com" confirma o retorno pelo link
    Então devo ver a mensagem "Este link já foi utilizado"

  @limite
  Esquema do Cenário: Link de confirmação vale por 48 horas
```

```
    Dado que a assinante "ana@exemplo.com" pediu para voltar à lista há <horas> horas
    Quando a assinante "ana@exemplo.com" confirma o retorno pelo link
    Então devo ver a mensagem "<mensagem>"
    E a assinante "ana@exemplo.com" deve <situacao>

    Exemplos:
      | horas | mensagem                    | situacao               |
      | 47    | Retorno à lista confirmado  | estar na lista         |
      | 48    | Retorno à lista confirmado  | estar na lista         |
      | 49    | Link de confirmação expirado | continuar fora da lista |
```

Feature 3 · Disparo respeita o descadastro

```
# language: pt
@US04
Funcionalidade: Disparo respeita o descadastro
  Como assinante
  Quero ter certeza de que, depois de sair, não recebo mais nada
  Para confiar que o descadastro funciona de verdade

  Contexto:
    Dado que existe a edição "Informativo #42" aprovada
    E que a assinante "ana@exemplo.com" está na lista

  @feliz
  Cenário: Quem está na lista recebe a edição
    Quando a edição "Informativo #42" é disparada
    Então a assinante "ana@exemplo.com" deve receber a edição "Informativo #42"

  @excecao
  Cenário: Quem se descadastrou não recebe a edição
    Dado que a assinante "bia@exemplo.com" se descadastrou
    Quando a edição "Informativo #42" é disparada
    Então a assinante "bia@exemplo.com" não deve receber a edição "Informativo #42"
    E a assinante "ana@exemplo.com" deve receber a edição "Informativo #42"

  @excecao
  Cenário: Retorno ainda não confirmado não volta a receber
    Dado que a assinante "bia@exemplo.com" se descadastrou
    E que a assinante "bia@exemplo.com" pediu para voltar à lista há 1 hora
    Quando a edição "Informativo #42" é disparada
    Então a assinante "bia@exemplo.com" não deve receber a edição "Informativo #42"
```

4. Action words (passos reutilizáveis)

Dado:

- `que a assinante "{email}" está na lista` — String
- `que a assinante "{email}" se descadastrou` — String
- `que a assinante "{email}" pediu para voltar à lista há {horas} hora(s)` — String, Integer
- `que a assinante "{email}" já confirmou o retorno` — String

Quando:

- `a assinante "{email}" se descadastra pelo link do e-mail` — String
- `a assinante "{email}" pede para voltar à lista` — String
- `a assinante "{email}" confirma o retorno pelo link` — String

- `a edição "{titulo}" é disparada` — String

Então:

- `a assinante "{email}" deve estar na lista` — String
- `a assinante "{email}" deve estar fora da lista` — String
- `a assinante "{email}" deve continuar fora da lista` — String
- `a assinante "{email}" deve receber o e-mail "{assunto}"` — String, String
- `a assinante "{email}" deve ter recebido {quantidade} e-mail(s) "{assunto}"` — String, Integer, String
- `a assinante "{email}" deve receber a edição "{titulo}"` — String, String
- `a assinante "{email}" não deve receber a edição "{titulo}"` — String, String
- `o e-mail "{assunto}" recebido por "{email}" deve trazer o link de descadastro` — String, String

Reutilizadas de outras histórias / comuns:

- `que existe a edição "{titulo}" aprovada` — US03
- `devo ver a mensagem "{mensagem}"` — comum.steps.js

Observação: o Cucumber diferencia Dado de Quando; por isso uma ação usada como pré-condição (ex.: "que insiro o texto") também existe como `Given`.

5. Código JavaScript (Cucumber.js)

features/step_definitions/us04-descadastro.steps.js

Steps da história. O estado fica no World (`this`), recriado a cada cenário; o domínio em memória e o relógio simulado (sem `sleep`) estão em `features/support/dominio.js` e `features/support/world.js` (Parte 7 deste documento).

```
// ============================================================================
// US04 · Sair e voltar à lista (RN06) — descadastro e duplo opt-in
// ============================================================================
const { Given, When, Then } = require('@cucumber/cucumber')
const assert = require('node:assert/strict')

// ---------------------------- DADO ----------------------------
Given('que a assinante {string} está na lista', function (email) {
  this.servico.inscrever(email)
})

Given('que a assinante {string} se descadastrou', function (email) {
  const r = this.servico.descadastrar(email)
  assert.ok(r.ok, r.mensagem)
})

Given('que a assinante {string} pediu para voltar à lista há {int} hora(s)', function (email, horas) {
  const r = this.servico.pedirRetorno(email)
  assert.ok(r.ok, r.mensagem)
  this.avancarHoras(horas)
})

Given('que a assinante {string} já confirmou o retorno', function (email) {
  const r = this.servico.confirmarRetorno(email)
```

```
  assert.ok(r.ok, r.mensagem)
})

// ---------------------------- QUANDO ----------------------------
When('a assinante {string} se descadastra pelo link do e-mail', function (email) {
  this.resultado = this.servico.descadastrar(email)
})

When('a assinante {string} pede para voltar à lista', function (email) {
  this.resultado = this.servico.pedirRetorno(email)
})

When('a assinante {string} confirma o retorno pelo link', function (email) {
  this.resultado = this.servico.confirmarRetorno(email)
})

When('a edição {string} é disparada', function (titulo) {
  assert.equal(this.edicao.titulo, titulo)
  this.resultado = this.servico.disparar(this.edicao)
})

// ---------------------------- ENTÃO ----------------------------
Then('a assinante {string} deve estar na lista', function (email) {
  assert.equal(this.servico.assinante(email).naLista, true)
})

Then('a assinante {string} deve estar fora da lista', function (email) {
  assert.equal(this.servico.assinante(email).naLista, false)
})

Then('a assinante {string} deve continuar fora da lista', function (email) {
  assert.equal(this.servico.assinante(email).naLista, false)
})

Then('a assinante {string} deve receber o e-mail {string}', function (email, assunto) {
  assert.ok(this.servico.emailsRecebidos(email, assunto).length > 0, `"${assunto}" não chegou para
${email}`)
})

Then('a assinante {string} deve ter recebido {int} e-mail(s) {string}', function (email, quantidade,
assunto) {
  assert.equal(this.servico.emailsRecebidos(email, assunto).length, quantidade)
})

Then('a assinante {string} deve receber a edição {string}', function (email, titulo) {
  assert.equal(this.servico.emailsRecebidos(email, titulo).length, 1)
})

Then('a assinante {string} não deve receber a edição {string}', function (email, titulo) {
  assert.equal(this.servico.emailsRecebidos(email, titulo).length, 0)
})

Then('o e-mail {string} recebido por {string} deve trazer o link de descadastro', function (assunto,
email) {
  const [mensagem] = this.servico.emailsRecebidos(email, assunto)
  assert.ok(mensagem, `"${assunto}" não chegou para ${email}`)
  assert.equal(mensagem.linkDescadastro, true)
})
```

6. Steps compartilhados

Passos usados por mais de uma história. As demais reutilizações entre histórias estão listadas na seção "Action words" de cada uma.

features/step_definitions/comum.steps.js

```
// ======================================================================
// STEPS COMPARTILHADOS ENTRE AS HISTÓRIAS (action words reutilizáveis)
// ======================================================================
const { Given, Then } = require('@cucumber/cucumber')
const assert = require('node:assert/strict')

Given('que estou autenticada como gestora', function () {
  this.servico.autenticar('gestora', 'Gestora')
})

Then('devo ver a mensagem {string}', function (esperada) {
  assert.ok(this.resultado, 'Nenhuma ação foi executada neste cenário')
  assert.equal(this.resultado.mensagem, esperada)
})
```

7. Suporte: World e domínio em memória

O `World` é recriado antes de cada cenário, garantindo independência entre eles. O domínio roda em memória com um relógio simulado — nenhum `sleep` real: "esperar 30 segundos" ou "faltam 2 minutos" apenas avançam o relógio.

features/support/world.js

```
// ========================================================================
// WORLD — recriado antes de cada cenário (nenhum estado vaza entre eles)
// ========================================================================
const { setWorldConstructor } = require('@cucumber/cucumber')
const { Servico, SEGUNDO, MINUTO, HORA } = require('./dominio')

class MundoDoNewsFlow {
  constructor () {
    this.servico = new Servico()
    this.edicao = null
    this.resultado = null
  }

  avancarSegundos (segundos) {
    this.servico.avancarRelogio(segundos * SEGUNDO)
  }

  avancarMinutos (minutos) {
    this.servico.avancarRelogio(minutos * MINUTO)
  }

  avancarHoras (horas) {
    this.servico.avancarRelogio(horas * HORA)
  }
}

setWorldConstructor(MundoDoNewsFlow)
```

features/support/dominio.js

```
// ========================================================================
// DOMÍNIO EM MEMÓRIA DO NEWSFLOW — relógio simulado, sem sleep real
// Regras implementadas: RN01, RN02, RN03, RN06
// ========================================================================
const SEGUNDO = 1000
const MINUTO = 60 * SEGUNDO
const HORA = 60 * MINUTO

// RN01 — Rascunho
const INTERVALO_AUTOSAVE_MS = 30 * SEGUNDO
const MAXIMO_VERSOES = 20
// RN03 — Agendamento
const ANTECEDENCIA_MINIMA_MIN = 15
const MAXIMO_REAGENDAMENTOS = 2
const LIMITE_CANCELAMENTO_MIN = 5
const FUSO = 'America/Sao_Paulo'
// RN06 — Descadastro
const VALIDADE_OPTIN_MS = 48 * HORA

const SITUACAO = {
  RASCUNHO: 'em rascunho',
```

```
  EM_REVISAO: 'aguardando revisão',
  APROVADA: 'aprovada'
}

const ok = (mensagem) => ({ ok: true, mensagem })
const falha = (mensagem) => ({ ok: false, mensagem })

class Edicao {
  constructor (titulo, corpo = '', autor = null) {
    this.titulo = titulo
    this.corpo = corpo
    this.autor = autor
    this.situacao = SITUACAO.RASCUNHO // RN01: toda edição nasce como rascunho
    this.versoes = [] // [{ numero, corpo }]
    this.proximoNumero = 1
    this.alterada = false
    this.ultimaAlteracao = 0
    this.comentarios = [] // [{ revisor, texto }]
    this.aprovadaPor = null
    this.disparo = null // { horario, fuso, reagendamentos, situacao }
  }
}

class Servico {
  constructor () {
    this.agora = 0
    this.usuario = null
    this.edicaoAtiva = null
    this.assinantes = new Map() // email -> { email, naLista, pedidoRetorno }
    this.caixas = new Map() // email -> [{ assunto, linkDescadastro, enviadoEm }]
  }

  // ---------------- relógio ----------------
  avancarRelogio (ms) {
    this.agora += ms
    const e = this.edicaoAtiva
    if (e && e.alterada && this.agora - e.ultimaAlteracao >= INTERVALO_AUTOSAVE_MS) {
      this.salvarVersao(e)
    }
  }

  // ---------------- sessão ----------------
  autenticar (papel, nome = null) {
    this.usuario = { papel, nome }
  }

  criarEdicao (titulo, corpo = '', autor = null) {
    this.edicaoAtiva = new Edicao(titulo, corpo, autor)
    return this.edicaoAtiva
  }

  // ========== US01 · RN01 — texto, versões e restauração ==========
  salvarVersao (edicao) {
    edicao.versoes.push({ numero: edicao.proximoNumero++, corpo: edicao.corpo })
    if (edicao.versoes.length > MAXIMO_VERSOES) edicao.versoes.shift()
    edicao.alterada = false
  }

  definirVersoes (edicao, quantidade) {
    edicao.versoes = []
    edicao.proximoNumero = 1
    for (let i = 0; i < quantidade; i++) this.salvarVersao(edicao)
  }
```

```
restaurarVersao (edicao, numero) {
  const versao = edicao.versoes.find((v) => v.numero === numero)
  if (!versao) return falha('Versão indisponível para restauro')
  edicao.corpo = versao.corpo // não apaga as versões seguintes
  return ok('Versão restaurada')
}

registrarAlteracao (edicao) {
  edicao.alterada = true
  edicao.ultimaAlteracao = this.agora
}

inserirTexto (edicao, texto) {
  if (texto.trim() === '') return falha('Digite algum texto para inserir')
  edicao.corpo += texto
  this.registrarAlteracao(edicao)
  return ok('Texto inserido')
}

substituirTrecho (edicao, trecho, novo) {
  if (!edicao.corpo.includes(trecho)) return falha('Trecho não encontrado')
  edicao.corpo = edicao.corpo.replace(trecho, novo)
  this.registrarAlteracao(edicao)
  return ok('Texto atualizado')
}

removerTrecho (edicao, trecho) {
  if (edicao.corpo === '') return falha('Não há texto para remover')
  if (!edicao.corpo.includes(trecho)) return falha('Trecho não encontrado')
  edicao.corpo = edicao.corpo.replace(trecho, '')
  this.registrarAlteracao(edicao)
  return ok('Texto removido')
}

// =========== US02 · RN02 — aprovação e reprovação ===========
enviarParaRevisao (edicao) {
  edicao.situacao = SITUACAO.EM_REVISAO
}

validarRevisor (edicao) {
  if (!this.usuario || this.usuario.papel !== 'revisor') {
    return falha('Somente revisores podem aprovar edições')
  }
  if (edicao.situacao !== SITUACAO.EM_REVISAO) {
    return falha('A edição não está aguardando revisão')
  }
  return null
}

aprovar (edicao) {
  const erro = this.validarRevisor(edicao)
  if (erro) return erro
  if (this.usuario.nome === edicao.autor) {
    return falha('O autor não pode aprovar a própria edição')
  }
  edicao.situacao = SITUACAO.APROVADA
  edicao.aprovadaPor = this.usuario.nome
  return ok('Edição aprovada')
}

reprovar (edicao, comentario) {
  const erro = this.validarRevisor(edicao)
```

```
  if (erro) return erro
  if (!comentario || comentario.trim() === '') {
    return falha('O comentário é obrigatório para reprovar')
  }
  edicao.situacao = SITUACAO.RASCUNHO
  edicao.comentarios.push({ revisor: this.usuario.nome, texto: comentario })
  return ok('Edição devolvida ao rascunho')
}

// =========== US03 · RN03 — agendar, reagendar e cancelar ===========
agendar (edicao, minutos) {
  if (edicao.situacao !== SITUACAO.APROVADA) {
    return falha('A edição precisa ser aprovada por um revisor')
  }
  if (minutos < ANTECEDENCIA_MINIMA_MIN) {
    return falha('O envio precisa de ao menos 15 minutos')
  }
  edicao.disparo = {
    horario: this.agora + minutos * MINUTO,
    fuso: FUSO,
    reagendamentos: 0,
    situacao: 'agendado'
  }
  return ok('Envio agendado')
}

reagendar (edicao, minutos) {
  const d = edicao.disparo
  if (!d || d.situacao !== 'agendado') return falha('Não há envio agendado para esta edição')
  if (d.reagendamentos >= MAXIMO_REAGENDAMENTOS) {
    return falha('O envio só pode ser reagendado 2 vezes')
  }
  if (minutos < ANTECEDENCIA_MINIMA_MIN) {
    return falha('O envio precisa de ao menos 15 minutos')
  }
  d.horario = this.agora + minutos * MINUTO
  d.reagendamentos++
  return ok('Envio reagendado')
}

cancelar (edicao) {
  const d = edicao.disparo
  if (!d || d.situacao !== 'agendado') return falha('Não há envio agendado para esta edição')
  if (this.minutosAteODisparo(edicao) < LIMITE_CANCELAMENTO_MIN) {
    return falha('O cancelamento só é permitido até 5 minutos antes do envio')
  }
  d.situacao = 'cancelado'
  return ok('Envio cancelado')
}

minutosAteODisparo (edicao) {
  return (edicao.disparo.horario - this.agora) / MINUTO
}

// =========== US04 · RN06 — descadastro e duplo opt-in ===========
inscrever (email) {
  this.assinantes.set(email, { email, naLista: true, pedidoRetorno: null })
  if (!this.caixas.has(email)) this.caixas.set(email, [])
  return this.assinantes.get(email)
}

assinante (email) {
  return this.assinantes.get(email) || this.inscrever(email)
```

```
  }

  enviarEmail (email, assunto) {
    // RN06: todo e-mail traz o link de descadastro
    this.caixas.get(email).push({ assunto, linkDescadastro: true, enviadoEm: this.agora })
  }

  emailsRecebidos (email, assunto) {
    return (this.caixas.get(email) || []).filter((m) => m.assunto === assunto)
  }

  descadastrar (email) {
    const a = this.assinante(email)
    if (!a.naLista) return falha('Você já está fora da lista')
    a.naLista = false // efeito imediato
    a.pedidoRetorno = null
    this.enviarEmail(email, 'Confirmação de descadastro')
    return ok('Descadastro realizado')
  }

  pedirRetorno (email) {
    const a = this.assinante(email)
    if (a.naLista) return falha('Você já está na lista')
    a.pedidoRetorno = { pedidoEm: this.agora, usado: false }
    this.enviarEmail(email, 'Confirme seu retorno à lista')
    return ok('Enviamos um link de confirmação válido por 48 horas')
  }

  confirmarRetorno (email) {
    const a = this.assinante(email)
    const pedido = a.pedidoRetorno
    if (!pedido) return falha('Não há pedido de retorno pendente')
    if (pedido.usado) return falha('Este link já foi utilizado')
    if (this.agora - pedido.pedidoEm > VALIDADE_OPTIN_MS) return falha('Link de confirmação expirado')
    pedido.usado = true
    a.naLista = true
    return ok('Retorno à lista confirmado')
  }

  disparar (edicao) {
    const destinatarios = [...this.assinantes.values()].filter((a) => a.naLista)
    destinatarios.forEach((a) => this.enviarEmail(a.email, edicao.titulo))
    return ok(`Edição enviada para ${destinatarios.length} assinante(s)`)
  }
}

module.exports = { Servico, Edicao, SITUACAO, SEGUNDO, MINUTO, HORA }
```

8. Configuração do projeto e execução

cucumber.js

```
// cucumber.js
module.exports = {
  default: {
    require: ['features/support/**/*.js', 'features/step_definitions/**/*.js'],
    format: ['progress', 'html:relatorios/cucumber-report.html'],
    publish: false
  }
}
```

package.json

```
{
  "name": "newsflow-bdd",
  "version": "1.0.0",
  "description": "NewsFlow (Vértice) — BDD com Cucumber.js: US01 a US04",
  "private": true,
  "scripts": {
    "test": "cucumber-js",
    "test:dry": "cucumber-js --dry-run"
  },
  "author": "Eduardo Vianna Meira (844387) e Vitor Alexandre (803371)",
  "license": "UNLICENSED",
  "devDependencies": {
    "@cucumber/cucumber": "^13.3.0"
  }
}
```

Estrutura de pastas

```
newsflow-bdd/
├── cucumber.js
├── package.json
├── README.md
├── docs/
│   ├── historias.md
│   ├── US01-criar-newsletter.md
│   ├── US02-aprovar-edicao.md
│   ├── US03-agendar-envio.md
│   └── US04-sair-e-voltar-a-lista.md
├── features/
│   ├── us01-inserir-texto.feature        us01-editar-texto.feature        us01-remover-texto.feature
│   ├── us02-aprovar-edicao.feature       us02-reprovar-edicao.feature     us02-envio-exige-
aprovacao.feature
│   ├── us03-agendar-envio.feature        us03-reagendar-envio.feature     us03-cancelar-envio.feature
│   ├── us04-descadastrar.feature         us04-voltar-a-lista.feature      us04-disparo-respeita-
descadastro.feature
│   ├── step_definitions/
│   │   ├── comum.steps.js
│   │   ├── us01-criar-newsletter.steps.js
│   │   ├── us02-aprovar-edicao.steps.js
│   │   ├── us03-agendar-envio.steps.js
│   │   └── us04-descadastro.steps.js
│   └── support/
│       ├── dominio.js
```

```
│       └── world.js
└── relatorios/cucumber-report.html
```

Saída de `npm test`

```
.....................................................................................................
.....................................................................................................
.....................................................

52 scenarios (52 passed)
276 steps (276 passed)
0m 0.188s (0m 0.028s executing your code)
```

Suíte verde: 52 cenários e 276 passos passando, sem passos pendentes nem cenários comentados.